

PA 回忆录

1. 表达式求值，是一个很大的问题，最后一道题，询问表达式求值是怎么实现的，似乎包含了一些复杂问题，比如怎么区分解引用和乘法；
2. 监视点，一个很短的问题，询问监视点怎么实现的。
3. 不太记得填表的操作了，好像印象里没有问具体某一条指令的细节，但是会问一条指令要填哪些文件，要改哪些东西。
4. 提问过这个，好像还问了 `_halt()` 的具体细节，是最难的一道题

我们对编译得到的可执行文件的行为进行简单的梳理：

1. 第一条指令从 `nexus-am/am/arch/x86-nemu/img/boot/start.S` 开始，设置好栈顶之后就跳转到 `nexus-am/am/arch/x86-nemu/src/trm.c` 的 `_trm_init()` 函数处执行。
2. 在 `_trm_init()` 中调用 `main()` 函数执行程序的主题功能。
3. 从 `main()` 函数返回后，调用 `_halt()` 结束运行。

阅读 `nexus-am/am/arch/x86-nemu/src/trm.c` 中的代码，你会发现只需要实现很少的API就可以支撑起程序在TRM上运行了：

- `_Area _heap` 结构用于指示堆区的起始和末尾
- `void _putc(char ch)` 用于输出一个字符
- `void _halt(int code)` 用于结束程序的运行
- `void _trm_init()` 用于进行TRM相关的初始化工作

最后来看看 `_halt()`。 `_halt()` 里面是一条内联汇编语句，内联汇编语句允许我们在C代码中嵌入汇编语句。这条指令和我们常见的汇编指令不一样(例如 `movl $1, %eax`)，它是直接通过指令的编码给出的，它只有一个字节，就是 `0xd6`。如果你在 `nemu/src/cpu/exec/exec.c` 中查看 `opcode_table`，你会发现，这条指令正是那条特殊的 `nemu_trap`！这其实也说明了为什么要通过编码来给出这条指令，如果你使用以下方式给出指令，汇编器将会报错：

```
asm volatile("nemu_trap" : : "a" (0))
```

因为这条特殊的指令是我们人为添加的，标准的汇编器并不能识别它。如果你查看objdump的反汇编结果，你会看到 `nemu_trap` 指令被标识为 `(bad)`，原因是类似的：objdump并不能识别我们人为添加的 `nemu_trap` 指令。`"a"(0)` 表示在执行内联汇编语句给出的汇编代码之前，先将 `0` 读入 `%eax` 寄存器。这样，这段汇编代码的功能就和 `nemu/src/cpu/exec/special.c` 中的helper函数 `nemu_trap()` 对应起来了。此外，`volatile` 是C语言的一个关键字，如果你想了解关于 `volatile` 的更多信息，请查阅相关资料。

5. IO 设备里提问重点是键盘，尤其是怎么判断按键按下了没有

键盘

键盘是最基本的输入设备. 一般键盘的工作方式如下: 当按下一个键的时候, 键盘将会发送该键的通码(make code); 当释放一个键的时候, 键盘将会发送该键的断码(break code). `nemu/src/device/keyboard.c` 模拟了 i8042 通用设备接口芯片的功能. 其大部分功能也被简化, 只保留了键盘接口. i8042 初始化时会注册 `0x60` 处的端口作为数据寄存器, 注册 `0x64` 处的端口作为状态寄存器. 每当用户敲下/释放按键时, 将会把相应的键盘码放入数据寄存器, 同时把状态寄存器的标志设置为 `1`, 表示有按键事件发生. CPU 可以通过端口 I/O 访问这些寄存器, 获得键盘码. 在 AM 中, 我们约定通码的值为 `断码 + 0x8000`.

如何检测多个键同时被按下

在游戏中, 很多时候需要判断玩家是否同时按下了多个键, 例如 RPG 游戏中的八方向行走, 格斗游戏中的组合招式等等. 根据键盘码的特性, 你知道这些功能是如何实现的吗?

实现 IOE(2)

实现 `_read_key()` 后, 在 NEMU 中运行 `keytest` 程序(在 `nexus-am/tests/keytest` 目录下). 如果你的实现正确, 在程序运行时弹出的新窗口中按下按键, 你将会看到程序输出相应的按键信息.

6. 我记得有这个问题, 两个 `Log()` 有什么区别, 两个 `Printf()` 有什么区别, 但是具体是 PA3 还是 PA4 记不清楚了。

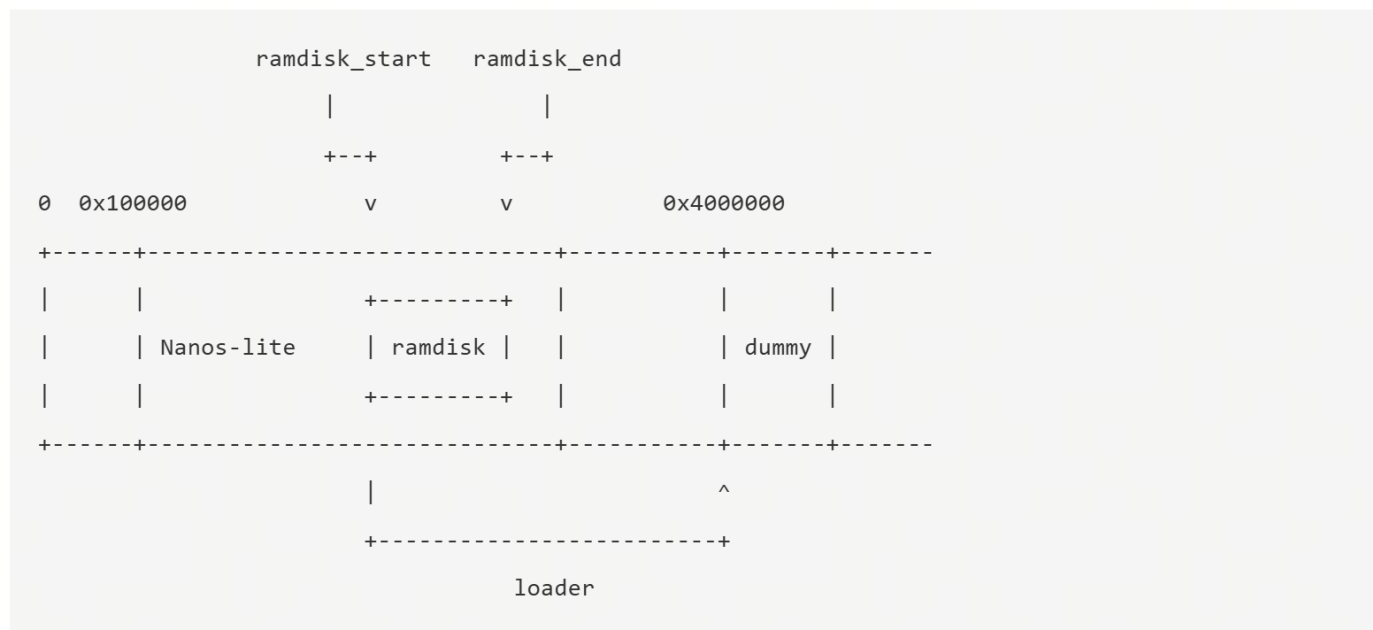
一开始, 在 `nanos-lite/src/main.c` 中所有与实验进度相关的宏都没有定义, 此时 Nanos-lite 的功能十分简单. 我们来简单梳理一下 Nanos-lite 目前的行为:

1. 通过 `Log()` 输出 hello 信息和编译时间. 需要说明的是, Nanos-lite 中定义的 `Log()` 宏并不是 NEMU 中定义的 `Log()` 宏. Nanos-lite 和 NEMU 是两个独立的项目, 它们的代码不会相互影响, 你在阅读代码的时候需要注意这一点. 在 Nanos-lite 中, `Log()` 宏通过 `klib` 中的 `printk()` 输出, 最终会调用 TRM 的 `_putc()`.
2. 初始化 ramdisk. 在一个完整的模拟器中, 程序应该存放在磁盘中. 但目前我们并没有实现磁盘的模拟, 因此先把 Nanos-lite 中的一段内存作为磁盘来使用. 这样的磁盘有一个专门的名字, 叫 `ramdisk`.
3. 调用 `init_device()` 对设备进行一些初始化操作. 目前 `init_device()` 会直接调用 `_ioe_init()`.
4. 调用 `loader()` 函数加载用户程序, 函数会返回用户程序的入口地址. 其中 `loader()` 函数并未实现, 我们会在下文进行说明.
5. 跳转到用户程序的入口执行.

7. 好像是这里提问了 `0x4000000` 放的是什么, 记不得了

为了回答剩下的问题, 我们首先需要了解可执行文件是如何组织的. 你应该已经在课堂上学习过ELF文件格式了, 它除了包含程序本身的代码和静态数据之外, 还包括一些用来描述它们的组织信息. 事实上, 我们的loader目前并没有必要去解析并加载ELF文件. 为了简化, `nanos-lite/Makefile` 中已经把用户程序运行所需要的代码和静态数据通过 `objcopy` 工具从ELF文件中抽取出来了, 整个ramdisk本身就已经存放了loader所需要加载的内容. 最后, "正确的内存位置", 也就是我们上文提到的约定好的 `0x4000000` 了.

所以, 目前的loader只需要做一件事情: 将ramdisk中从0开始的所有内容放置在 `0x4000000`, 并把这个地址作为程序的入口返回即可. 我们把这个简化了的loader称为raw program loader. 我们通过内存布局来理解loader目前需要做的事情:



8. 要自己知道 Nanos-lite 运行流程

一开始, 在 `nanos-lite/src/main.c` 中所有与实验进度相关的宏都没有定义, 此时Nanos-lite的功能十分简单. 我们来简单梳理一下Nanos-lite目前的行为:

1. 通过 `Log()` 输出hello信息和编译时间. 需要说明的是, Nanos-lite中定义的 `Log()` 宏并不是NEMU中定义的 `Log()` 宏. Nanos-lite和NEMU是两个独立的项目, 它们的代码不会相互影响, 你在阅读代码的时候需要注意这一点. 在Nanos-lite中, `Log()` 宏通过 `klib` 中的 `printk()` 输出, 最终会调用TRM的 `_putc()`.
2. 初始化ramdisk. 在一个完整的模拟器中, 程序应该存放在磁盘中. 但目前我们并没有实现磁盘的模拟, 因此先把Nanos-lite中的一段内存作为磁盘来使用. 这样的磁盘有一个专门的名字, 叫ramdisk.
3. 调用 `init_device()` 对设备进行一些初始化操作. 目前 `init_device()` 会直接调用 `_ioe_init()`.
4. 调用 `loader()` 函数加载用户程序, 函数会返回用户程序的入口地址. 其中 `loader()` 函数并未实现, 我们会在下文进行说明.
5. 跳转到用户程序的入口执行.

9. 这个地方是问了汇编代码含义的, 我记得:

Navy-apps已经为用户程序准备好了系统调用的接口了. `navy-apps/libs/libos/src/nanos.c` 中定义的 `_syscall_()` 函数已经蕴含着上述过程:

```
int _syscall_(int type, uintptr_t a0, uintptr_t a1, uintptr_t a2) {
    int ret;
    asm volatile("int $0x80": "=a"(ret): "a"(type), "b"(a0), "c"(a1), "d"(a2));
    return ret;
}
```

上述内联汇编会先把系统调用的参数依次放入 `%eax`, `%ebx`, `%ecx`, `%edx` 四个寄存器中, 然后执行 `int $0x80` 手动触发一个特殊的异常. 操作系统捕获这个异常之后, 发现是一个系统调用, 就会调出相应的处理函数进行处理, 处理结束后设置好返回值, 然后返回到上述的内联汇编中. 内联汇编最后从 `%eax` 寄存器中取出系统调用的返回值, 并返回给调用该接口的函数, 告知其系统调用执行的情况(如是否成功等).

我们可以在GNU/Linux下编写一个程序, 来手工触发一次 `write` 系统调用:

```
const char str[] = "Hello world!\n";

int main() {
    asm volatile ("movl $4, %eax;"      // system call ID, 4 = SYS_write
                  "movl $1, %ebx;"      // file descriptor, 1 = stdout
                  "movl $str, %ecx;"    // buffer address
                  "movl $13, %edx;"     // length
                  "int $0x80");

    return 0;
}
```

10. 提问了 trap frame 结构体

重新组织TrapFrame结构体

你的任务如下:

- 实现 `pusha` 指令, 你需要注意压栈的顺序, 更多信息请查阅i386手册.
- 理解trap frame形成的过程, 然后重新组织 `nexus-am/am/arch/x86-nemu/include/arch.h` 中定义的 `_RegSet` 结构体的成员, 使得这些成员声明的顺序和 `nexus-am/am/arch/x86-nemu/src/trap.S` 中构造的trap frame保持一致.

实现正确之后, `irq_handle()` 以及后续代码就可以正确地使用trap frame了. 重新在Nanos-lite上运行dummy程序, 你会看到在 `nanos-lite/src/irq.c` 中的 `do_event()` 函数中触发了BAD TRAP:

```
[src/irq.c,5,do_event] {kernel} system panic: Unhandled event ID = 8
```

11. 一个很长的问题，PA3 是怎么处理异常的，得给助教展示函数调用流程

触发异常

为了测试是否已经成功准备IDT, 我们还需要真正触发一次异常, 看是否正确地跳转到目标地址. 具体的, 你需要在NEMU中实现 `raise_intr()` 函数(在 `nemu/src/cpu/intr.c` 中定义) 来模拟上文提到的i386中断机制的处理过程:

```
void raise_intr(uint8_t NO, vaddr_t save_addr) {  
    /* TODO: Trigger an interrupt/exception with ``NO``.  
     * That is, use ``NO`` to index the IDT.  
     */  
  
}
```

需要注意的是:

- PA不涉及特权级的切换, 查阅i386手册的时候你不需要关心和特权级切换相关的内容.
- 通过IDTR中的地址对IDT进行索引的时候, 需要使用 `vaddr_read()` .
- PA中不实现分段机制, 没有CS寄存器的概念. 但为了在QEMU中顺利进行differential testing, 我们还是在cpu结构体中添加一个CS寄存器, 并在 `restart()` 函数中将其初始化为 `8` .
- 由于中断机制需要对EFLAGS进行压栈, 为了配合differential testing, 我们还需要在 `restart()` 函数中将EFLAGS初始化为 `0x2` .
- 执行 `int` 指令后保存的 `EIP` 指向的是 `int` 指令的下一条指令, 这有点像函数调用, 具体细节可以查阅i386手册.
- 你需要在 `int` 指令的helper函数中调用 `raise_intr()` , 而不要把中断机制的代码放在 `int` 指令的helper函数中实现, 因为在后面我们会再次用到 `raise_intr()` 函数.

12. 异常这里我记得有挺多小问题，自己仔细看看吧

13. 提问了怎么实现分页机制

然后你需要对 `vaddr_read()` 和 `vaddr_write()` 函数作少量修改. 以 `vaddr_read()` 为例, 修改后如下:

```
uint32_t vaddr_read(vaddr_t addr, int len) {
    if (data cross the page boundary) {
        /* this is a special case, you can handle it later. */
        assert(0);
    }
    else {
        paddr_t paddr = page_translate(addr);
        return paddr_read(paddr, len);
    }
}
```

你需要理解分页地址转过的过程, 然后编写 `page_translate()` 函数. 另外由于我们不打算实现保护机制, 在 `page_translate()` 函数的实现中, 你务必使用assertion检查页目录项和页表项的present位, 如果发现了一个无效的表项, 及时终止NEMU的运行, 否则调试将会异常困难. 这通常是由于你的实现错误引起的, 请检查实现的正确性. 再次提醒, 只有进入保护模式并开启分页机制之后才会进行页级地址转换. 为了让differential testing机制正确工作, 你还需要实现分页机制中accessed位和dirty位的功能.

最后提醒一下页级地址转换时出现的一种特殊情况. 由于i386并没有严格要求数据对齐, 因此可能会出现数据跨越虚拟页边界的情况, 例如一条很长的指令的首字节在一个虚拟页的最后, 剩下的字节在另一个虚拟页的开头. 如果这两个虚拟页被映射到两个不连续的物理页, 就需要进行两次页级地址转换, 分别读出这两个物理页中需要的字节, 然后拼接起来组成一个完成的数据返回. MIPS作为一种RISC架构, 指令和数据都严格按照4字节对齐, 因此不会发生这样的情况, 否则MIPS CPU将会抛出异常, 可见软件灵活性和硬件复杂度是计算机科学中又一对tradeoff. 不过根据KISS法则, 你现在可以暂时不实现这种特殊情况的处理, 在判断出数据跨越虚拟页边界的情况之后, 先使用 `assert(0)` 终止NEMU, 等到真的出现这种情况的时候再进行处理.

在NEMU中实现分页机制

根据上述的讲义内容, 在NEMU中实现i386分页机制, 如有疑问, 请查阅i386手册.

14. 我只记得这里问了很多, 甚至还有 `_switch` 的细节

目前 `schedule()` 只需要总是切换到第一个用户进程即可, 即 `pcb[0]`. 注意它的上下文是在加载程序的时候通过 `_umake()` 创建的, 在 `schedule()` 中才决定要切换到它, 然后在ASYE的 `asm_trap()` 中才真正地恢复这一上下文. 在 `schedule()` 返回之前, 还需要切换到新进程的虚拟地址空间. 这样, 等到从异常处理的代码返回之后, 我们就已经正确地在仙剑奇侠传的虚拟地址空间中运行仙剑奇侠传的代码了!

实现上下文切换

根据讲义的上述内容, 实现以下功能:

- PTE的 `_umake()` 函数
- Nanos-lite的 `schedule()` 函数
- Nanos-lite收到 `_EVENT_TRAP` 事件后, 调用 `schedule()` 并返回其现场
- 修改ASYE中 `asm_trap()` 的实现, 使得从 `irq_handle()` 返回后, 先将栈顶指针切换到新进程的陷阱帧, 然后才根据陷阱帧的内容恢复现场, 从而完成上下文切换的本质操作

实现成功后, Nanos-lite就可以通过内核自陷触发上下文切换的方式运行仙剑奇侠传了.

15. 时钟中断怎么实现

在NEMU中, 我们只需要添加时钟中断这一种中断就可以了. 由于只有一种中断, 我们也不需要通过中断控制器进行中断的管理, 直接让时钟中断连接到CPU的INTR引脚即可, 我们也约定时钟中断的中断号是 32 . 时钟中断通过 `nemu/src/device/timer.c` 中的 `timer_intr()` 触发, 每10ms触发一次. 触发后, 会调用 `dev_raise_intr()` 函数(在 `nemu/src/cpu/intr.c` 中定义). 你需要:

- 在cpu结构体中添加一个 `bool` 成员 `INTR` .
- 在 `dev_raise_intr()` 中将INTR引脚设置为高电平.
- 在 `exec_wrapper()` 的末尾添加轮询INTR引脚的代码, 每次执行完一条指令就查看是否有硬件中断到来:

```
#define TIMER_IRQ 32

if (cpu.INTR & cpu.eflags.IF) {
    cpu.INTR = false;
    raise_intr(TIMER_IRQ, cpu.eip);
    update_eip();
}
```

- 修改 `raise_intr()` 中的代码, 在保存EFLAGS寄存器后, 将其IF位置为 0 , 让处理器进入关中断状态.

在软件上, 你还需要:

- 在ASYS中增加时钟中断的支持, 将时钟中断打包成 `_EVENT_IRQ_TIME` 事件.
- Nanos-lite收到 `_EVENT_IRQ_TIME` 事件之后, 直接调用 `schedule()` 进行进程调度, 同时也可以去掉系统调用之后调用的 `schedule()` 代码了.
- 为了可以让处理器在运行用户进程的时候响应时钟中断, 你还需要修改 `_umake()` 的代码, 在构造现场的时候, 设置正确的EFLAGS.

添加时钟中断

根据讲义的上述内容, 添加相应的代码来实现真正的分时多任务.